

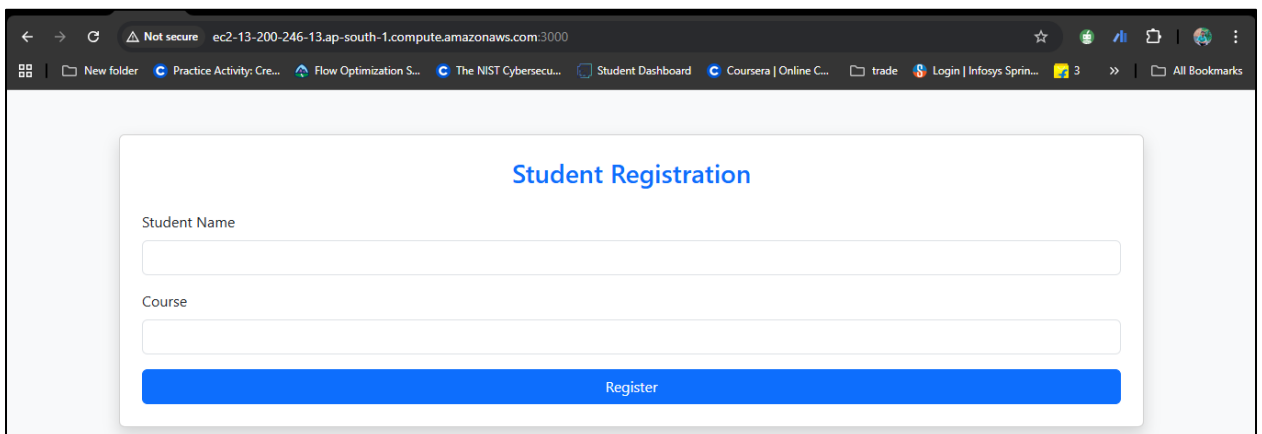
1. Deploy Your flask backend and express frontend in amazon single ec2 instance.

```
(venv) ubuntu@ip-172-31-15-145:~/simple-ec2-app/backend$ # Replace 1234 with the actual PID from your output
sudo kill -9 3935
(venv) ubuntu@ip-172-31-15-145:~/simple-ec2-app/backend$ python app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.15.145:5000
Press CTRL+C to quit
client_loop: send disconnect: Connection reset
PS C:\Users\pawar\OneDrive\Desktop\DevOps\AWS>
```

```
ubuntu@ip-172-31-15-145:~/simple-ec2-app/frontend$ npm start

> student-frontend@1.0.0 start
> node server.js

Frontend running on port 3000
```



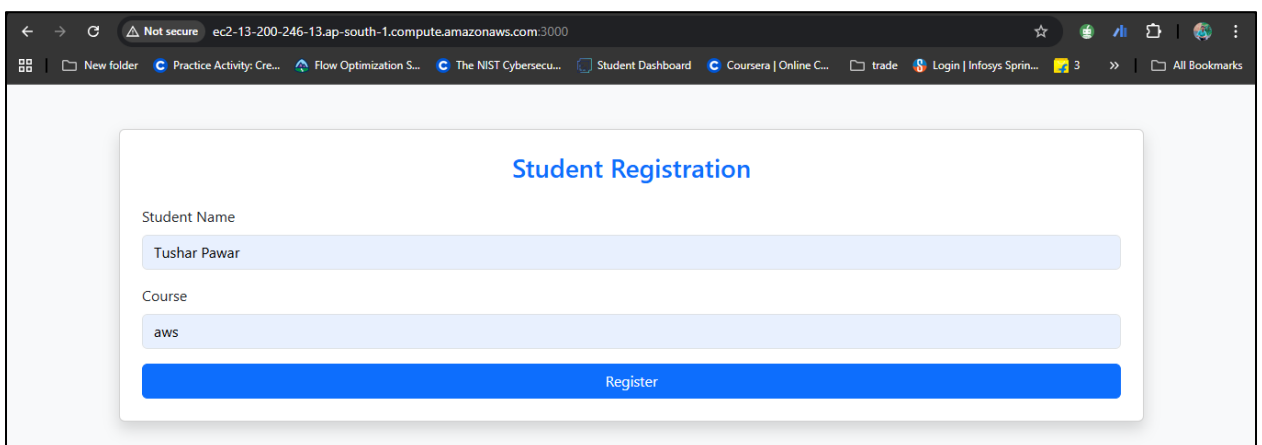
ec2-13-200-246-13.ap-south-1.compute.amazonaws.com:3000

Student Registration

Student Name

Course

Register



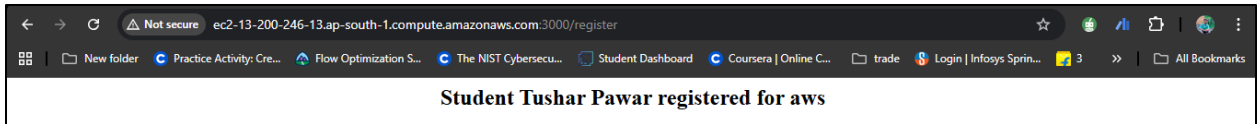
ec2-13-200-246-13.ap-south-1.compute.amazonaws.com:3000

Student Registration

Student Name

Course

Register



2. Deploy Your flask backend and express frontend in separate ec2 instances.

```
C:\Users\pawar\OneDrive\Desktop\DevOps\student-app>ssh -i "window.pem"
ubuntu@13.200.235.203
The authenticity of host '13.200.235.203 (13.200.235.203)' can't be est
ablished.
ED25519 key fingerprint is SHA256:JhvbnZqIwy18tSjCEDvr6t+Xe8BdNE8VurR1
wHoHi0.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? ye
s
Warning: Permanently added '13.200.235.203' (ED25519) to the list of kn
own hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

```
C:\Users\pawar\OneDrive\Desktop\DevOps\student-app>ssh -i "window.pem"
ubuntu@13.203.199.19
The authenticity of host '13.203.199.19 (13.203.199.19)' can't be estab
lished.
ED25519 key fingerprint is SHA256:XgWQn34H2XorefJI00/jzjCyIMEWjPiM7rnZx
5xLLPM.
This key is not known by any other names.
Are you sure you want to continue connecting (yes/no/[fingerprint])? ye
s
Warning: Permanently added '13.203.199.19' (ED25519) to the list of kno
wn hosts.
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1015-aws x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro
```

← → ↻ Not secure ec2-13-203-199-19.ap-south-1.compute.amazonaws.com:3000

☰ | 📁 New folder 🔄 Practice Activity: Cre... 🔄 Flow Optimization S... 🔄 The NIST Cybersecu... 📄 Student Dashboard 🔄 Coursera | Online C... 📁 trade 🔄 Login | Infos...

Student Registration

```
(venv) ubuntu@ip-172-31-12-30:~/backend$ python3 app.py
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.31.12.30:5000
Press CTRL+C to quit
172.31.4.185 - - [01/Jan/2026 16:03:46] "POST /api/contact HTTP/1.1" 200 -
```

← → ↻ Not secure ec2-13-203-199-19.ap-south-1.compute.amazonaws.com:3000/submit

☰ | 📁 New folder 🔄 Practice Activity: Cre... 🔄 Flow Optimization S... 🔄 The NIST Cybersecu... 📄 Student Dashboard 🔄 Coursera | Online C... 📁 trade 🔄 Login | Infos...

Form submitted successfully ✓

3. Deploy Your flask backend and express frontend Docker Container using aws ecr, ecs and vpc services

```
C:\AWS\my-app>aws ecr get-login-password --region ap-south-1 | docker login --username AWS --password-stdin
outh-1.amazonaws.com
Login Succeeded
```

Building Flask backend Docker image locally

```
C:\AWS\my-app\backend>docker build -t flask-backend .
[+] Building 2.8s (11/11) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 210B                          0.0s
=> [internal] load metadata for docker.io/library/python:3.11-s 2.4s
=> [auth] library/python:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                             0.0s
=> => transferring context: 2B                                 0.0s
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:1dd3dca 0.0s
=> => resolve docker.io/library/python:3.11-slim@sha256:1dd3dca 0.0s
=> [internal] load build context                             0.0s
=> => transferring context: 93B                                0.0s
=> CACHED [2/5] WORKDIR /app                                 0.0s
=> CACHED [3/5] COPY requirements.txt .                       0.0s
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements. 0.0s
=> CACHED [5/5] COPY . .                                     0.0s
=> exporting to image                                       0.1s
```

Building Express frontend Docker image locally

```
C:\AWS\my-app\frontend>docker build -t express-frontend .
[+] Building 2.6s (11/11) FINISHED          docker:desktop-linux
=> [internal] load build definition from Dockerfile          0.0s
=> => transferring dockerfile: 161B                          0.0s
=> [internal] load metadata for docker.io/library/node:18-slim 2.2s
=> [auth] library/node:pull token for registry-1.docker.io 0.0s
=> [internal] load .dockerignore                             0.0s
=> => transferring context: 2B                                 0.0s
=> [1/5] FROM docker.io/library/node:18-slim@sha256:f9ab18e354e 0.0s
=> => resolve docker.io/library/node:18-slim@sha256:f9ab18e354e 0.0s
=> [internal] load build context                             0.0s
=> => transferring context: 192B                                0.0s
=> CACHED [2/5] WORKDIR /app                                 0.0s
=> CACHED [3/5] COPY package.json .                          0.0s
=> CACHED [4/5] RUN npm install                             0.0s
=> CACHED [5/5] COPY . .                                     0.0s
=> exporting to image                                       0.1s
=> => exporting layers                                         0.0s
=> => exporting manifest sha256:c807aaaf9a4f01df7685bb299e598fb2 0.0s
=> => exporting config sha256:8b59dc90ce436e83894ad45f80f9c0d14 0.0s
=> => exporting attestation manifest sha256:a4d79e75a226fc0272d 0.0s
=> => exporting manifest list sha256:29f0c47cbe774bbedc33845600 0.0s
```

 Creating ECR repository for Flask backend in Mumbai region

 Creating ECR repository for Express frontend in Mumbai region

```
C:\AWS\my-app>aws ecr create-repository --repository-name flask-backend --region ap-south-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:039638668084:repository/flask-backend",
    "registryId": "039638668084",
    "repositoryName": "flask-backend",
    "repositoryUri": "039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-backend",
    "createdAt": "2026-01-09T06:55:30.112000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    },
    "encryptionConfiguration": {
      "encryptionType": "AES256"
    }
  }
}

C:\AWS\my-app>aws ecr create-repository --repository-name express-frontend --region ap-south-1
{
  "repository": {
    "repositoryArn": "arn:aws:ecr:ap-south-1:039638668084:repository/express-frontend",
    "registryId": "039638668084",
    "repositoryName": "express-frontend",
    "repositoryUri": "039638668084.dkr.ecr.ap-south-1.amazonaws.com/express-frontend",
    "createdAt": "2026-01-09T06:55:46.393000+05:30",
    "imageTagMutability": "MUTABLE",
    "imageScanningConfiguration": {
      "scanOnPush": false
    }
  },
}
```

 Tagging backend Docker image for ECR repository

 Pushing Flask backend image to AWS ECR - Upload complete!

```
C:\AWS\my-app\backend>docker tag flask-backend:latest 039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest

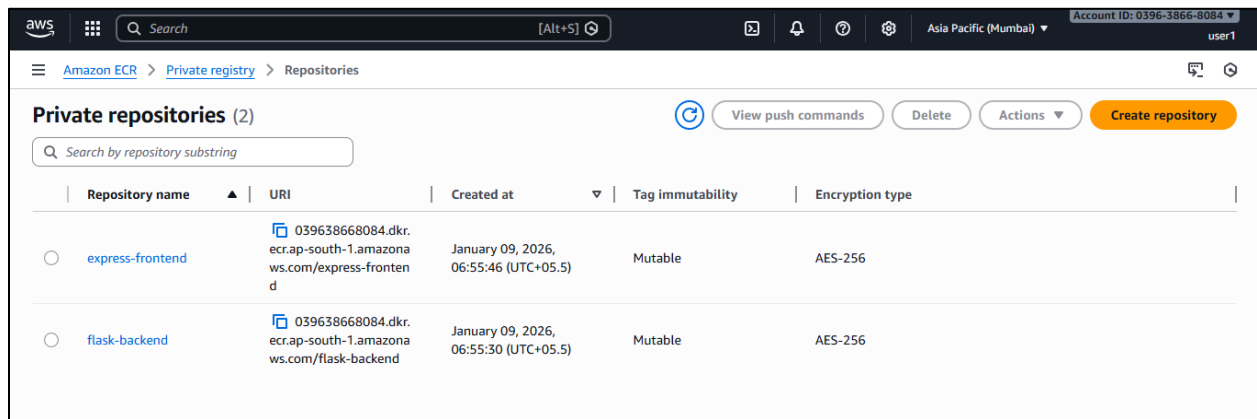
C:\AWS\my-app\backend>docker push 039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-backend:latest
The push refers to repository [039638668084.dkr.ecr.ap-south-1.amazonaws.com/flask-backend]
49c86427b2e3: Pushed
6e1d340982ef: Pushed
7cc13cb22d92: Pushed
1473863bb010: Pushed
b591fe1ed2b8: Pushed
654a090213c0: Pushed
02d7611c4eae: Pushed
fa20452c5ebf: Pushed
6463d6d2b0fe: Pushed
latest: digest: sha256:1fd862dd25e768266b020750e5f879af50ae83b0922b0b7fe09c8d01a975615f size: 856
```

 Tagging frontend Docker image for ECR repository

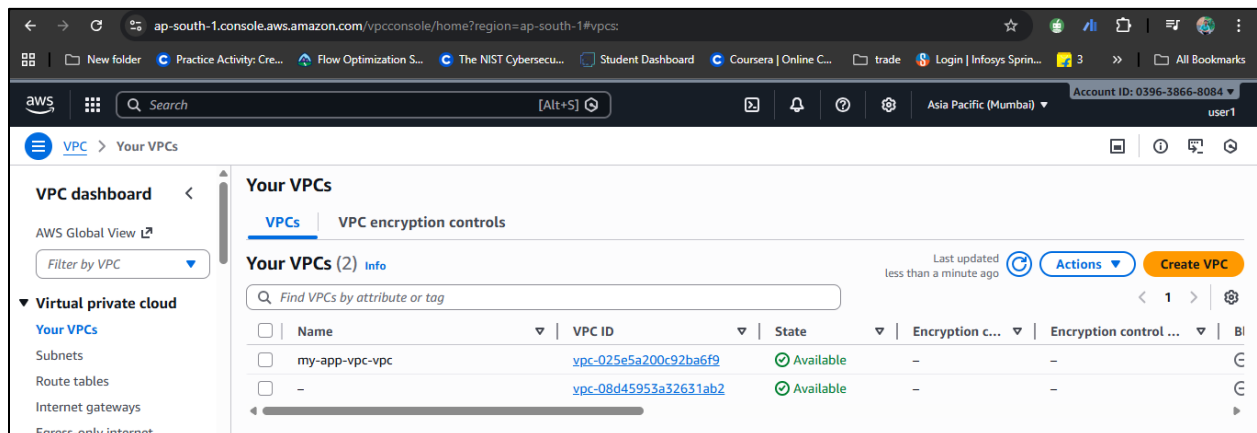
 Pushing Express frontend image to AWS ECR - Upload complete!

```
C:\AWS\my-app\frontend>docker tag express-frontend:latest 039638668084.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest

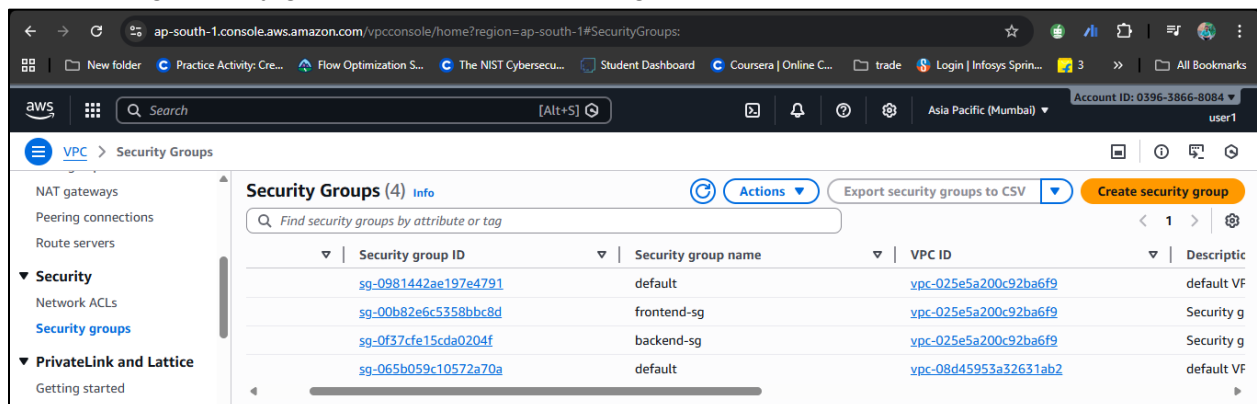
C:\AWS\my-app\frontend>docker push 039638668084.dkr.ecr.ap-south-1.amazonaws.com/express-frontend:latest
The push refers to repository [039638668084.dkr.ecr.ap-south-1.amazonaws.com/express-frontend]
1ad9456d9c00: Pushed
29e572e44201: Pushed
61320b01ae5e: Pushed
b98d3ae1ab80: Pushed
557944f3b2dc: Pushed
e3e2c76bf6b5: Pushed
c34b4c634246: Pushed
8c994cf49dd1: Pushed
c768ab8cba73: Pushed
b1831021e35a: Pushed
latest: digest: sha256:29f0c47cbe774bbdec338456007dc6a0d812b2b38cc74ba1d40ccde1bd60d6f3 size: 856
```



- 🌐 Creating VPC with CIDR 10.0.0.0/16 for our application network
- 📍 Creating public subnet 1 in ap-south-1a availability zone
- 📍 Creating public subnet 2 in ap-south-1b availability zone
- 🌐 Creating Internet Gateway to enable internet access for containers
- 🔗 Attaching Internet Gateway to VPC
- 🔗 Associating both public subnets with the route table



- 🔑 Creating security group for backend - allowing port 5000
- 🔑 Creating security group for frontend - allowing ports 80 and 3000



⚙️ Creating ECS Cluster with AWS Fargate (serverless containers)

Amazon Elastic Container Service > Clusters

frontend-service has been deployed successfully. [View service](#)

Clusters (2) Info Last updated January 9, 2026, 08:18 (UTC+5:30) [Create cluster](#)

Search clusters

By default, we only load up to 1,000 clusters at a time.

Cluster	Services	Tasks	Container instances	CloudWatch monitoring
my-app-cluster	2	0 Pending 2 Running	0 EC2	Default
student-app-cluster	0	No tasks running	0 EC2	Default

📄 Creating ECS Task Definition for Flask backend

📄 Creating ECS Task Definition for Express frontend

Amazon Elastic Container Service > Task definitions

frontend-service has been deployed successfully. [View service](#)

Task definitions (4) Info Last updated January 9, 2026, 08:18 (UTC+5:30) [Deploy](#) [Create new revision](#) [Create new task definition](#)

Filter task definitions Filter status: Active

Task definition	Status of last revision
backend-task	Active
frontend-task	Active
student-backend-task	Active
student-frontend-task	Active

🚢 Creating ECS Service for frontend with 1 desired task

🔧 Configuring frontend networking with public IP enabled

✅ Backend service deployed - waiting for task to start running...

✅ Frontend service deployed - waiting for task to start running...

CloudFormation > Stacks

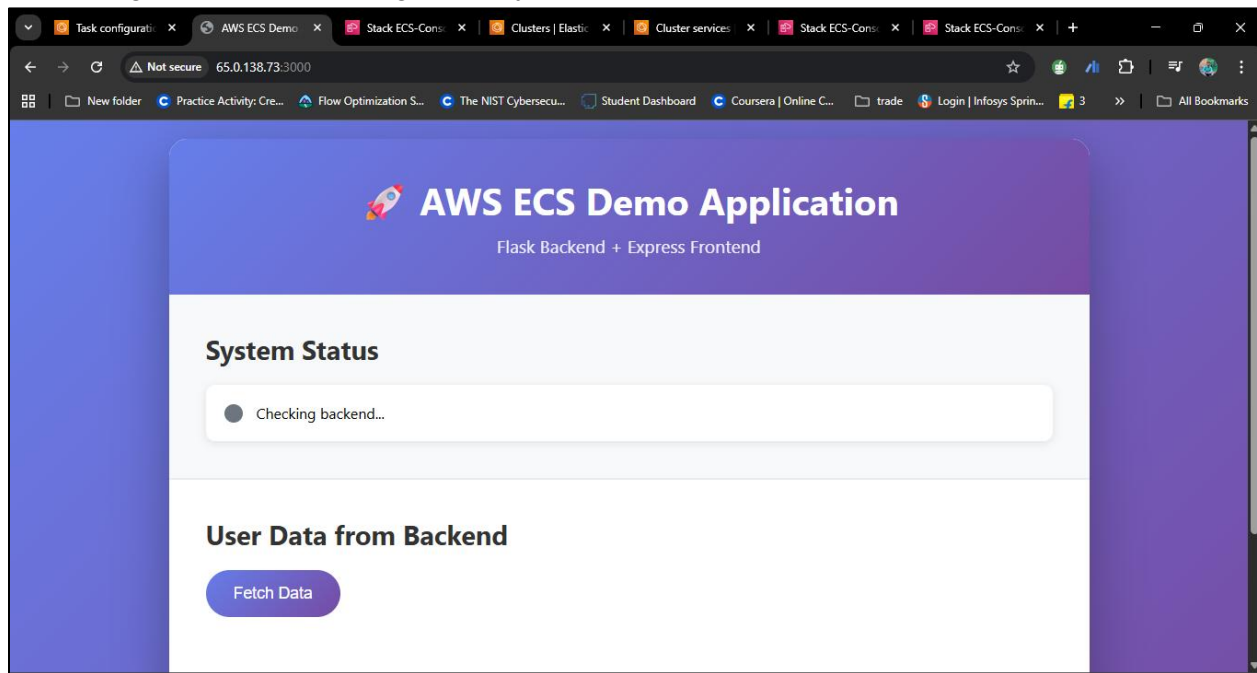
Stacks (3) [Delete](#) [Update stack](#) [Stack actions](#) [Create stack](#)

Filter status: Active [View nested](#)

Search by stack name

Stack name	Status	Created time	Description
ECS-Console-V2-Service-frontend-service-my-app-cluster-cb3bb94f	CREATE_COMPLETE	2026-01-09 07:57:56 UTC+0530	The template used to create : ECS Service from the ECS Console.
ECS-Console-V2-Service-backend-service-my-app-cluster-bdbce3e5	CREATE_COMPLETE	2026-01-09 07:44:46 UTC+0530	The template used to create : ECS Service from the ECS Console.
Infra-ECS-Cluster-my-app-cluster-544afc4c	CREATE_COMPLETE	2026-01-09 07:32:08 UTC+0530	The template used to create : ECS Cluster from the ECS Console.

🌐 Accessing frontend application via public IP on port 3000 and ✅ Application loaded!
Beautiful gradient UI is working perfectly!



- 🔍 Checking backend health status - System shows "Online"
- 🇮🇳 Clicking "Fetch Data" button to retrieve user data from backend
- 🎯 Success! Backend API returned user data and displayed beautifully

